

OE-EC604C — Object Oriented Programming

Semester VI • 3L:0T:0P • 3 Credits

Module 8 Exam Notes: Files and Streams

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	C++ File Streams Hierarchy	2
1.1	The Stream Class Hierarchy	2
2	Opening & Closing Files	2
2.1	File Open Methods	2
2.2	File Modes	2
3	File Pointers and Seek Operations	3
3.1	Seek and Tell Functions	3
4	Binary & Random Access Operations	3
4.1	Object Writing using <code>write()</code> and <code>read()</code>	3
4.2	Random Access Database Record Update	4

1. C++ File Streams Hierarchy

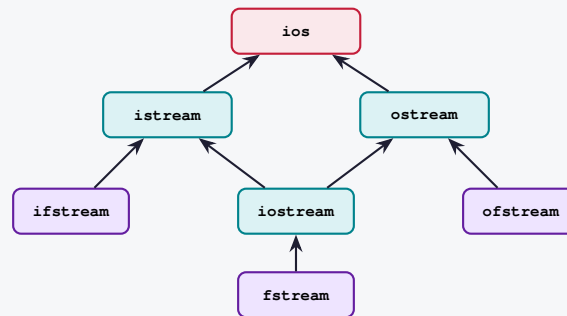
C++ models file input and output operations using a system of classes inherited from basic stream classes.

1.1 The Stream Class Hierarchy

All stream classes are derived from a common virtual base class `ios`. The main classes used for file operations are:

- **ifstream**: Input file stream. Derived from `istream`, used for reading files.
- **ofstream**: Output file stream. Derived from `ostream`, used for creating and writing files.
- **fstream**: Input/Output file stream. Derived from `iostream`, used for simultaneous reading and writing operations.

C++ Stream Class Hierarchy Topology



2. Opening & Closing Files

Files can be opened in two ways: using the constructor or using the `open()` method.

2.1 File Open Methods

1. **Using Constructor:** Best when managing a single file throughout the stream's lifetime.

```
ofstream outFile("data.txt"); // Opens file automatically (1)
```

2. **Using `open()` method:** Best when reusing a single stream object to access multiple files sequentially.

```
ifstream inFile; inFile.open("data.txt"); (2)
```

2.2 File Modes

File modes specify the access permissions and options for opening files.

File Mode Constant	Description
<code>ios::in</code>	Opens the file for reading (default for <code>ifstream</code>).
<code>ios::out</code>	Opens the file for writing (default for <code>ofstream</code>). Erases existing content.
<code>ios::app</code>	Append mode. All write operations occur at the end of the file.
<code>ios::ate</code>	Opens the file and moves the pointer to the end of the file.
<code>ios::binary</code>	Opens the file in binary mode (instead of default text mode).

3. File Pointers and Seek Operations

Each file stream object maintains file pointers that keep track of reading or writing locations.

3.1 Seek and Tell Functions

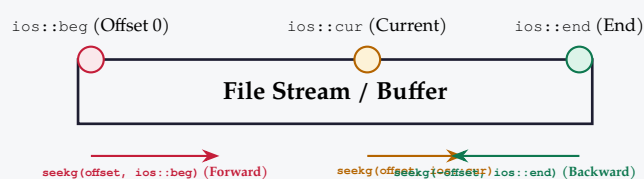
- `seekg()`: Moves the **get** (read) pointer to a new position.
- `seekp()`: Moves the **put** (write) pointer to a new position.
- `tellg()`: Returns the current offset position of the get pointer.
- `tellp()`: Returns the current offset position of the put pointer.

Seek Syntax Parameters

```
stream.seekg(offset, direction); (3)
```

Where direction can be: `ios::beg` or `ios::cur` or `ios::end` (4)

File Pointer Position and Offset Directions



4. Binary & Random Access Operations

By default, files are opened in text mode. Binary mode allows reading and writing complete structures or objects to files directly without parsing.

4.1 Object Writing using `write()` and `read()`

```
file.write(reinterpret_cast<char*>(&object), sizeof(object)); (5)
```

```
file.read(reinterpret_cast<char*>(&object), sizeof(object)); (6)
```

Code Example: Reading and writing objects to binary files

```
#include <iostream>
#include <fstream>
#include <cstring>
class Student {
private:
    char name[30];
    int roll;
public:
    Student() : roll(0) { name[0] = '\0'; }
    Student(const char* n, int r) {
        strcpy(name, n);
        roll = r;
    }
    void show() {
        std::cout << "Roll: " << roll << ", Name: " << name << std::endl;
    }
};

void saveRecord() {
    Student s1("Biraj", 101);
    std::ofstream file("student.dat", std::ios::out | std::ios::binary);
    file.write(reinterpret_cast<char*>(&s1), sizeof(s1));
    file.close();
}

void loadRecord() {
    Student s2;
    std::ifstream file("student.dat", std::ios::in | std::ios::binary);
    file.read(reinterpret_cast<char*>(&s2), sizeof(s2));
    s2.show(); // Prints: Roll: 101, Name: Biraj
    file.close();
}
```

4.2 Random Access Database Record Update

Using `seekp()` and index logic, we can update specific object slots directly inside a binary database without reading the whole file.

Code Example: Random Access database record update

```
#include <iostream>
#include <fstream>
struct Account {
    int accNum;
    double balance;
};

void updateBalance(int recordId, double newBalance) {
    std::fstream file("account.dat", std::ios::in | std::ios::out | std::ios::binary);
    Account acc;

    // Jump to the specific record offset using sizeof
    int offset = (recordId - 1) * sizeof(Account);
    file.seekp(offset, std::ios::beg);

    acc.accNum = recordId;
    acc.balance = newBalance;

    // Rewrite record
    file.write(reinterpret_cast<char*>(&acc), sizeof(Account));
    file.close();
}
```

Quick Revision — Module VIII

Files and Streams | OE-EC604C

Key File I/O Syntax

Open Write (Trunc)	<code>ofstream out("a.txt", ios::out); // overrides</code>
Open Append	<code>ofstream out("a.txt", ios::app); // appends</code>
Binary Open	<code>fstream file("a.dat", ios::in ios::out ios::binary);</code>
Seek from Beginning	<code>file.seekg(100, ios::beg); // jumps to offset 100</code>
Seek Backward (End)	<code>file.seekp(-50, ios::end); // jumps 50 bytes before EOF</code>
Get Current Position	<code>int pos = file.tellg(); // returns read offset</code>

One-Line Definitions

- **File Stream:** The logical interface that connects the program to physical files on the disk.
- **ifstream:** A file stream class specialized in reading data from disk files.
- **ofstream:** A file stream class specialized in writing data to disk files.
- **fstream:** A stream class that supports both input and output file operations.
- **File Mode:** Settings that configure file permissions (read, write, binary, append) when a file is opened.
- **Get Pointer:** The read pointer that keeps track of the next index byte to read from a file.
- **Put Pointer:** The write pointer that keeps track of the next index byte to write to a file.
- **tellg():** A file function returning the current index offset of the read (get) pointer.
- **seekg():** A file function used to reposition the get pointer to a specified index location.
- **Binary Mode:** Opening a file to write raw memory bytes directly, bypassing text formatting.

Frequently Asked / Viva Questions

Q1. What is the difference between opening a file using constructor vs. the open() method?

The constructor automatically opens the file and is clean for single files. The

Frequently Asked / Viva Questions (contd.)

`open()` function allows opening multiple files sequentially using the same stream object.

Q2. Explain the difference between the `ios::app` and `ios::ate` file modes.

Both open the file with pointers positioned at the end. However, `ios::app` forces all write operations to occur at the end of the file, whereas `ios::ate` allows shifting the pointer to write anywhere in the file.

Q3. Why is `reinterpret_cast<char*>` mandatory in `read()` and `write()` functions?

These functions are designed to accept a stream of bytes (`char*`). We must cast our class object address to a byte pointer for binary transfer.

Q4. What is random file access and how is it implemented in C++?

Random access allows reading/writing at any arbitrary location in the file without processing it sequentially. It is implemented using `seekg()`, `seekp()`, `tellg()`, and `tellp()`.

Q5. What happens if we attempt to open a non-existent file for reading?

The file stream fails to open, and its error flag is set. We can verify this state using the `fail()` function or checking the stream's truth value: `if(!file)`.

Q6. Why must we close files explicitly before exiting the program?

Closing files flushes the stream buffers (saving unwritten data from RAM to disk) and releases system file handles.

Q7. Can we open a single file for simultaneous reading and writing?

Yes, using the `fstream` class and combining file modes: `fstream file("data.dat", ios::in | ios::out | ios::binary)`.

Q8. What is the purpose of the `eof()` function in C++ streams?

It returns `true` if the get pointer has reached the end-of-file (EOF) during a read operation.

Q9. How does text mode file access differ from binary mode file access?

Text mode translates line endings (like converting `\n` to `\r\n` on Windows) and parses values. Binary mode copies raw memory bytes bit-by-bit without any translations.

Q10. Which class acts as the root class for all file stream classes?

The root class is `ios` (specifically, `ios_base` is the grandparent class, and `ios` inherits from it).

Q11. How can we determine the total size of a file in bytes using seek functions?

We can seek to the end of the file and use `tellg()` to query the pointer offset: `file.seekg(0, ios::end); int size = file.tellg();`

Frequently Asked / Viva Questions (contd.)

Q12. What is a file stream buffer and why is it used?

It is a temporary block of memory in RAM that buffers read/write bytes to minimize slow disk access operations, improving performance.

Comparison: Sequential File Access vs. Random File Access

Feature	Sequential File Access	Random File Access
Pointer Mechanics	Pointers move step-by-step from beginning to end.	Pointers can jump directly to any arbitrary offset.
Speed	Slower for large databases (must read intermediate records).	Extremely fast (access time is constant, $O(1)$, using offsets).
File Functions	Uses standard operators like <code><<</code> , <code>>></code> , or <code>getline</code> .	Uses functions like <code>seekg()</code> , <code>seekp()</code> , <code>tellg()</code> , <code>tellp()</code> .
Common Modes	Text mode (comma/space-separated values).	Binary mode (fixed-size struct or class records).
Use Case	Reading configuration files or writing log lines.	Modifying specific records in a large binary database.